

02

JOUR 2

Fonctions avancées, Tools & Agents spécialisés

1 Actions avancées & customisation

2 Les Tools OpenAI

3 Agents & sous-agents

4 Exécution sécurisée du code

5 Mises en pratique

Rappel express du Jour 1



Codex : l'outil OpenAI pour le dev, à comparer avec une grille de critères



Un bon prompt : objectif · contraintes · style · format attendu



Un contexte persistant (AGENTS.md) évite de tout répéter



Les actions font gagner du temps sur les tâches courantes



Le diff est votre outil de contrôle : on relit avant d'accepter



Au programme aujourd'hui



Tools & function calling



Agents spécialisés



Exécution sécurisée du code

Personnaliser les actions et créer des templates



Personnaliser les actions

Adapter les actions aux habitudes de votre équipe.



Créer des templates d'instructions

Transformer un long prompt en modèle réutilisable.



Automatiser les workflows

Des résultats plus homogènes, quel que soit l'utilisateur.

TEMPLATE « NOUVELLE FEATURE » (ILLUSTRATIF)

Contexte

Le module concerné, le besoin métier.

Étapes

Plan → code → tests → documentation.

Contraintes

Conventions, dépendances autorisées.

Livrables

Code, tests, README mis à jour.

Organiser ses prompts en équipe



Librairie interne

Un endroit partagé pour les prompts qui ont fait leurs preuves.



Patterns

Des modèles de formulation pour les situations récurrentes.



Conventions d'équipe

Des règles communes : nommage, validation, évolution.

EXEMPLE D'ORGANISATION (ILLUSTRATIF)

```
mon-projet/
```

```
  prompts/
```

```
    revue-de-code.md
```

```
    generer-tests.md
```

```
    nouvelle-feature.md
```

```
    refactor.md
```

```
  README.md
```

Versionnés comme du code : relus, améliorés, partagés.

Configurations avancées : permissions et restrictions



Permissions

Qui ou quoi peut faire quelle action : lecture seule, ou lecture et écriture ?



Accès au code

Quels dossiers et quels fichiers sont visibles et modifiables ?
Exclure les secrets.



Restrictions

Ce qui est interdit, ou qui exige une validation humaine avant exécution.

Principe du moindre privilège

Donner uniquement les droits nécessaires à la tâche : on peut toujours élargir ensuite.

Function calling : donner des outils au modèle



Le modèle n'exécute rien lui-même : il demande un appel, et c'est votre code qui agit.

Créer son propre tool : déclarer un schéma

EXEMPLE ILLUSTRATIF • JSON

```
{
  "name": "get_order_status",
  "description": "Retourne le statut
    d'une commande à partir de
    son identifiant",
  "parameters": {
    "type": "object",
    "properties": {
      "order_id": { "type": "string" }
    },
    "required": ["order_id"]
  }
}
```



Nom

Identifie l'outil.



Description

Permet au modèle de décider quand l'utiliser.



Paramètres

Types, champs obligatoires, valeurs autorisées.

Le schéma ne remplace pas la validation : votre code vérifie toujours les arguments.

Cas d'usage des tools



Migrations

Génération automatique de migrations de base de données.



SQL sécurisé

Des requêtes encadrées : en lecture seule, avec des paramètres contrôlés.



Données

Manipulation de données : transformer, filtrer, agréger.



Build

Automatisation de tâches de build : compilation, vérifications.

Chaque outil reçoit le minimum nécessaire : un outil de consultation ne doit pas pouvoir supprimer.

Qu'est-ce qu'un agent spécialisé ?



Agent assistant

Il propose

Il analyse, explique, recommande. Vous décidez et vous appliquez. Le risque est faible.



Agent exécutant

Il agit

Il modifie des fichiers, lance des commandes, dans le cadre des permissions accordées. Plus de gain, plus d'encadrement.

Un agent spécialisé = un rôle précis + des instructions + des outils adaptés. Commencez par des assistants, puis donnez de l'autonomie.

Orchestration multi-agents



Écosystème d'agents : on assemble les rôles utiles au projet (architecte, codeur, relecteur, mais aussi tests, documentation...).

Bonnes pratiques et limites actuelles



Bonnes pratiques

- Un rôle clair par agent
- Des permissions minimales
- Une validation humaine aux points clés
- Commencer petit, puis élargir



Limites actuelles

- Plus de tokens consommés, donc plus de coût
- Conflits possibles si plusieurs agents écrivent en même temps
- Une erreur peut se propager d'un agent à l'autre
- Une supervision humaine reste nécessaire

Exécuter du code de façon sécurisée



Code Interpreter

Le modèle exécute du code dans un environnement isolé.



Sandbox

Un environnement contraint qui limite ce que le code peut faire.



Execution plan

Le plan d'action, relu et validé avant application.

NIVEAUX DE SANDBOX DANS CODEX

read-only

Inspection uniquement

workspace-write

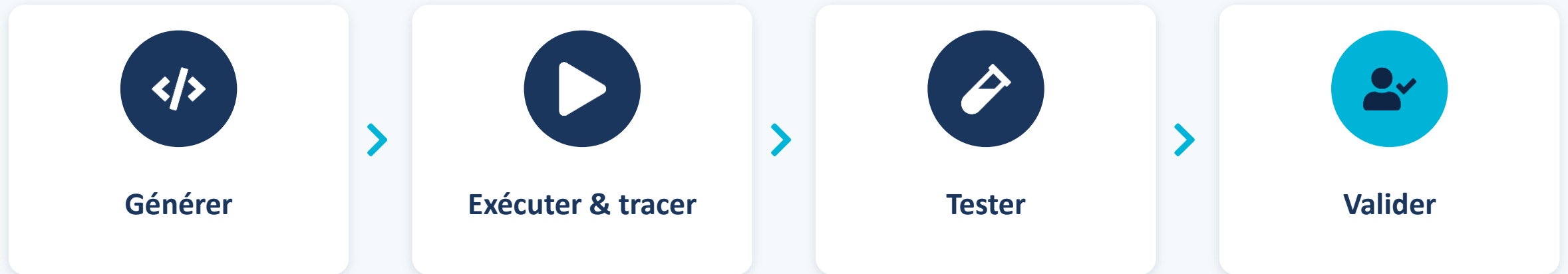
Lecture, édition dans l'espace de travail, commandes locales

danger-full-access

Sans restriction

Source : documentation officielle. Noms susceptibles d'évoluer.

Produire du code fiable, réduire les hallucinations



Si un problème apparaît : corriger, puis recommencer la boucle

Tests

Le code fait-il ce qu'on attend ?

Benchmarks

Quelles sont ses performances ?

Validations automatiques

Les contrôles sont faits à chaque modification.

Mises en pratique • Jour 2

1

Action personnalisée



Créer une action pour organiser une feature.

2

Tool personnalisé



Ajouter à Codex un tool branché sur une API interne.

3

Duo d'agents



Architecte + développeur pour implémenter une nouvelle fonctionnalité.

4

Plan de refactor



Générer un plan de refactor automatisé sur un projet existant.

Réflexes : contexte précis • permissions minimales • tests • relecture

Bilan de votre projet fil rouge



Génération de code

Projet structuré, module, tests et documentation.



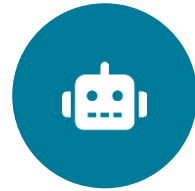
Automatisation

Action personnalisée et tool branché sur une API.



Refactor assisté

Plan de refactor, modifications contrôlées via les diffs.



Agents dédiés

Duo architecte + développeur sur une nouvelle fonctionnalité.

À chaque étape : prompt structuré · permissions maîtrisées · tests · relecture humaine

Vos acquis : les 6 objectifs atteints



Structurer un prompt efficace pour générer du code fiable



Configurer Codex dans son IDE et gérer un projet multi-fichiers



Utiliser les actions pour documenter, tester et refactorer



Créer des Tools personnalisés avec function calling



Mettre en place des agents spécialisés pour automatiser le développement



Sécuriser l'exécution du code et valider les résultats

Passez à l'action dans votre entreprise



Démarrer petit

Un cas d'usage simple et à faible risque : documentation ou tests d'un module existant.



Formaliser le contexte

Écrire le contexte persistant du projet et une petite librairie de prompts partagée.



Poser vos règles

Définir les permissions, les validations humaines et l'exigence de tests.

Les interfaces changent, les principes restent : bien formuler, bien cadrer, bien vérifier.

Merci !

Des questions ?

L'IA accélère votre travail.
Vous gardez la maîtrise du code.

Daouda DIOP

Senior Data Scientist & Expert IA Générative